

qcode

User Manual

A terminal-first AI coding assistant.
This guide shows how to use qcode day to day.

Version 1.0 - September 27, 2026

Usage guide for people using qcode.

No setup internals or code architecture inside.

In this manual: install once, tour the screen, ask and queue work,
switch models, use agents, plan, skills, sessions, browser control, and settings.

How to use this manual

Read chapters 1-3 to start. Keep chapters 4 and 15 nearby as a command reference. Chapters 5-12 are task guides you can read when you need them: models, agents, planning, skills, memory, web, sessions, and phone/browser control.

- Words in ``code style`` are things you type, for example ``/model`` or ``qcode --demo``.
- Figures are example screens. Your colors and sizes may differ slightly.
- The prompt line shows ``>`` when idle, ``(Plan)>`` in Plan mode, and ``(Queue)>`` while busy.
- Esc goes back one level. Ctrl+C cancels without applying changes.

Tip

Try everything first with: `qcode --demo "show me how qcode works"`. Nothing is installed and your files are untouched.

1. Getting started

Install and first run

qcode is one program you run in your terminal. Build it once, then start it in any project folder.

```
# Build from source (needs Go 1.22+)
make build
./bin/qcode
```

Pick the assistant you want to use. The two common choices are a local model (Ollama) or a hosted model (OpenAI or OpenCode Go).

```
# Local model (default provider)
ollama pull qwen2.5-coder:7b
qcode --model qwen2.5-coder:7b

# Hosted model
export OPENAI_API_KEY=...
qcode --provider openai --model gpt-5

# No setup tour
qcode --demo "show me how qcode works"
```

Tip

If you only want to learn the keys and screens, start with `--demo`. It runs a scripted tour with fake tools and queued prompts.

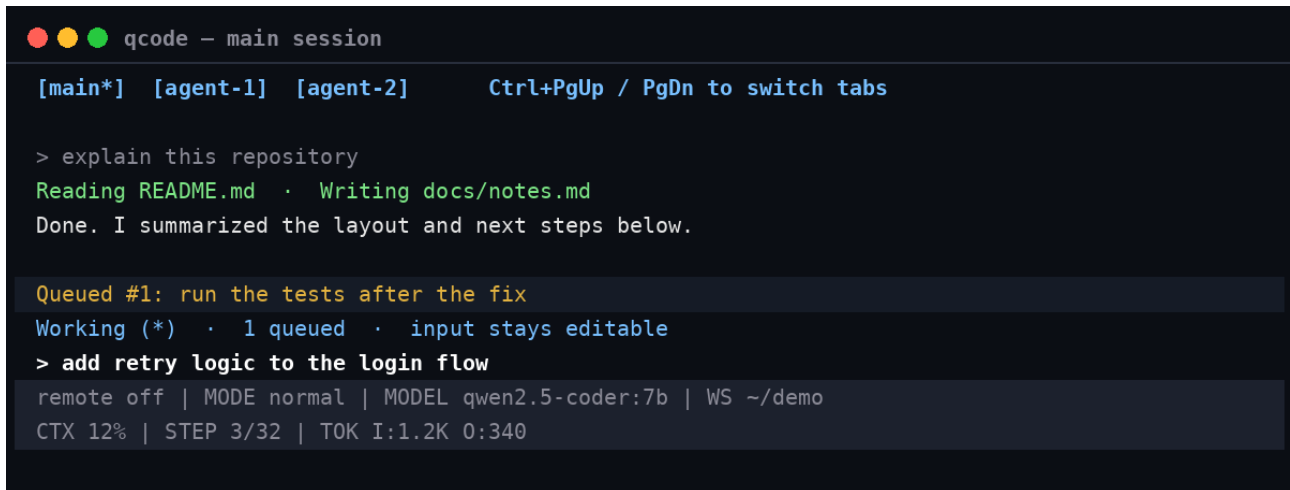
Update

Update the installed program from inside qcode when a new release is available.

```
qcode update
qcode update --arch arm64
```

2. Touring the terminal

qcode fills the terminal but keeps your normal scrollbar. Tabs sit at the top, the conversation fills the middle, and the prompt plus status bar stay pinned at the bottom.



```

qcode - main session

[main*] [agent-1] [agent-2]      Ctrl+PgUp / PgDn to switch tabs

> explain this repository
Reading README.md · Writing docs/notes.md
Done. I summarized the layout and next steps below.

Queued #1: run the tests after the fix
Working (*) · 1 queued · input stays editable
> add retry logic to the login flow
remote off | MODE normal | MODEL qwen2.5-coder:7b | WS ~/demo
CTX 12% | STEP 3/32 | TOK I:1.2K O:340
  
```

Figure 1: Main screen. Tabs on top, transcript in the middle, queue notice, editable prompt, and status bar at the bottom.

What you see

- Tabs: `main` is always there. Extra agents appear as new tabs (up to 20).
- Transcript: streamed answers with Markdown, tables, and tool activity like `Reading ...` or `Writing ...`.
- Queue area: appears only while busy. It lists waiting prompts in order.
- Prompt: `>` idle, `(Plan)>` planning, `(Queue)>` working. You can keep typing while work runs.
- Status bar: remote, mode, model, folder, context %, step, and tokens. Hide parts with `/statusline`.

Keys you will use daily

- Type `/` to see matching commands. Type more to filter, Tab to complete, Esc to close.
- Left/Right, Home/End move in the line. Ctrl+Left/Right jumps by word. Ctrl+W deletes a word. Ctrl+A/E jump to ends.
- PageUp/PageDown scrolls history, even while the agent works. Scrolling pauses the live view; PageDown to the bottom resumes it.
- Alt+Q expands a long queue so you can scroll it; Alt+Q again folds it back.
- Ctrl+C cancels the running prompt. Queued prompts wait their turn.
- Set NO_COLOR=1 to turn off colors. Set QCODE_ASCII=1 for plain-ASCII borders.

3. Asking, queueing, and history

Type a request and press Enter. While one request runs, you can type the next one. It is marked `Queued #N` and runs automatically in order.

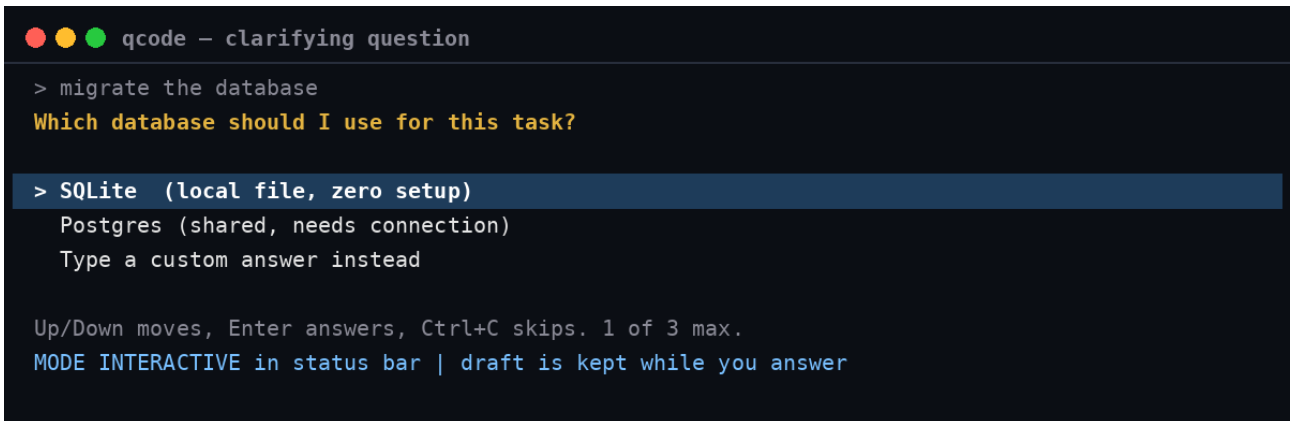
- One prompt runs at a time per agent. Other agents are not blocked.
- The tab and task line show how many prompts are waiting.
- Use ``/history`` to find an old prompt. Pick one to re-read its final answer.
- Use ``/new`` to clear the current conversation without restarting qcode.
- Use ``/compact`` when a long session feels slow. It shortens stored context.

Tip

Example flow: ask ``explain this repository``, then while it works queue ``run the tests after the fix``. Both complete in order without you waiting.

When qcode asks you back (`/interactive`)

Normal work can pause to ask one focused question when an ambiguity would otherwise need several broad searches. Turn this on per agent with ``/interactive on``, off with ``/interactive off``, or check the setting with ``/interactive``. It is off by default. Add ``interactive = true`` to `config.toml` to enable it for new terminal sessions.



```

qcode - clarifying question

> migrate the database
Which database should I use for this task?

> SQLite (local file, zero setup)
Postgres (shared, needs connection)
Type a custom answer instead

Up/Down moves, Enter answers, Ctrl+C skips. 1 of 3 max.
MODE INTERACTIVE in status bar | draft is kept while you answer

```

Figure 2b: A clarifying question. Pick a suggestion or type your own. At most 3 questions per prompt.

- One question at a time, with suggested choices or your own custom answer. At most 3 distinct questions per submitted prompt.
- Your answer becomes context for that agent only. It is not auto-saved; use ``/learn`` to keep it.
- The status bar shows `MODE INTERACTIVE` while enabled. Your half-typed draft is saved and restored around the question.
- Questions for other tabs wait on those tabs. `Ctrl+C` skips the question. The browser remote can also answer.
- One-shot prompts, piped input, and non-terminal runs never wait; qcode just proceeds with available context.

Tip

Turn it on when tasks are ambiguous (which database, which scope). Leave it off for strict hands-off runs.

4. Slash commands at a glance

Type ``/help`` to list commands or ``/help <name>`` for one command. The leading ``/`` is optional in

the name.

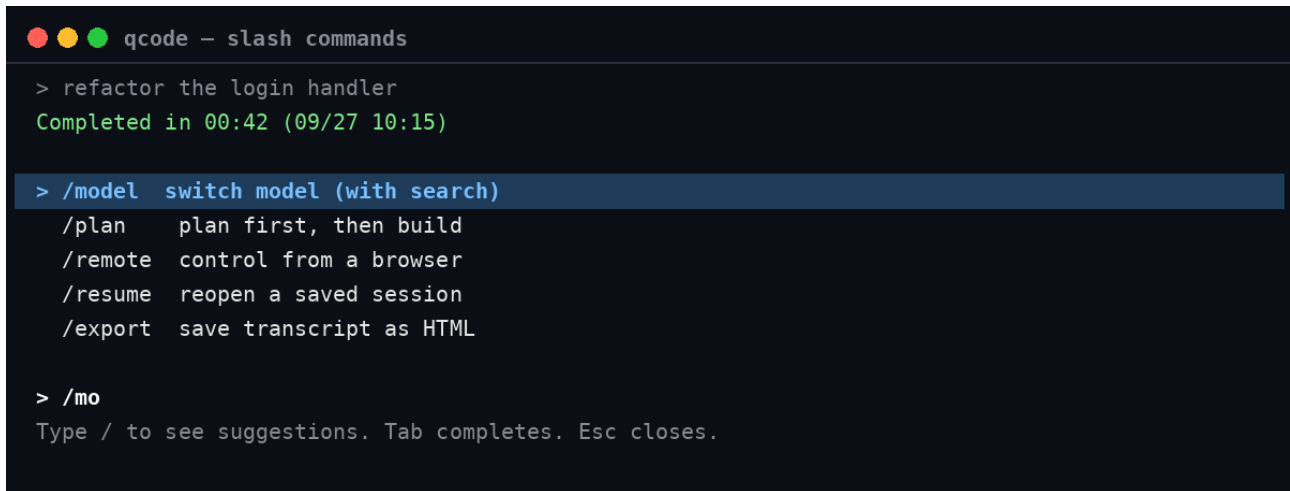
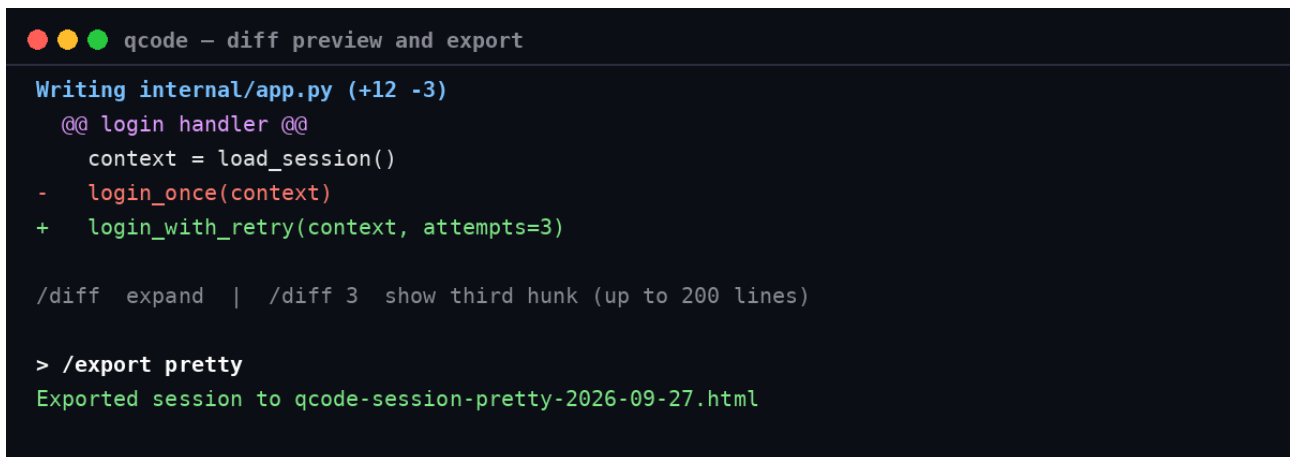
A screenshot of a terminal window titled 'qcode - slash commands'. The terminal shows a sequence of commands and their outputs. The first command is '> refactor the login handler', which results in 'Completed in 00:42 (09/27 10:15)'. The second command is '> /model switch model (with search)', which lists several slash commands: '/plan plan first, then build', '/remote control from a browser', '/resume reopen a saved session', and '/export save transcript as HTML'. The third command is '> /mo', which is partially shown. At the bottom, a message says 'Type / to see suggestions. Tab completes. Esc closes.'

Figure 2: Type / to filter commands. Tab completes the first match.

Command	What it does for you
/help	List commands or explain one command.
/new	Clear conversation and token totals for this tab.
/resume	Reopen a saved session for this folder.
/clear	Redraw the welcome banner and tool summary.
/history	Search completed prompts and re-read one answer.
/diff [N]	Expand the latest file-change preview.
/verbose	Show detailed telemetry for this session.
/maxsteps [N]	Show or change the per-request step limit.
/statusline	Show, hide, or reset status bar parts.
/compact	Shorten conversation context manually.
/export	Save transcript as readable or full HTML.
/quit, /exit	Leave qcode.

Diff previews

When qcode writes or edits a file you see a short numbered preview with added and removed lines. Use ``/diff`` to expand it up to a safe limit. The model still receives plain text.



```

qcode - diff preview and export

Writing internal/app.py (+12 -3)
  @@ login handler @@
    context = load_session()
-   login_once(context)
+   login_with_retry(context, attempts=3)

/diff expand | /diff 3 show third hunk (up to 200 lines)

> /export pretty
Exported session to qcode-session-pretty-2026-09-27.html

```

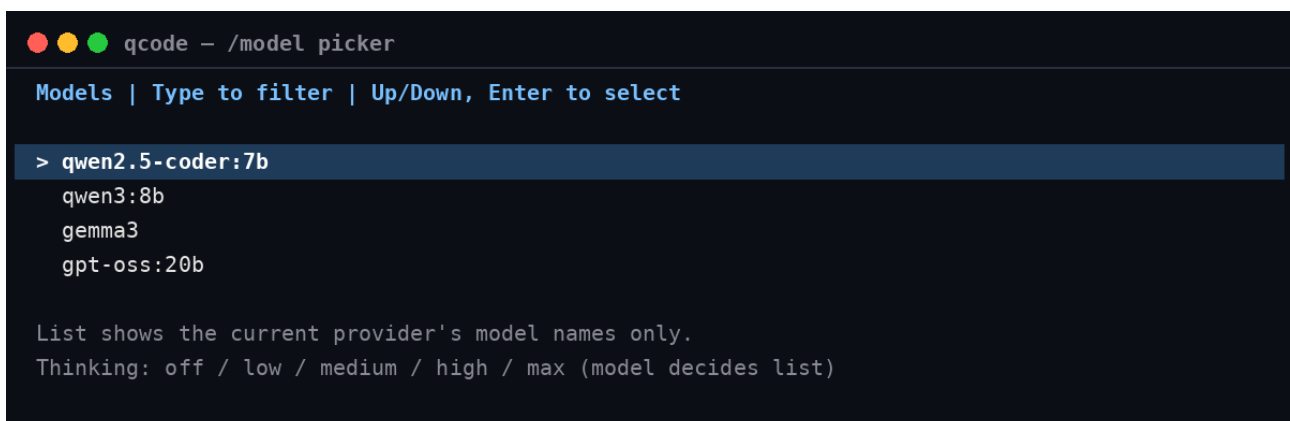
Figure 3: Numbered change preview plus `/export pretty` saving a readable HTML copy.

Exports

- `/export` or `/export pretty`: one tab per agent, prompt cards that reveal answers. Best for sharing and reading.
- `/export raw`: full styled transcript including tool activity, thinking, errors, and diffs.
- Files are saved as `qcode-session-<mode>-<timestamp>.html`. A checkbox reveals detailed tool events.

5. Choosing models and providers

Use `/model` to switch the assistant for the current tab. The list shows model names from your current provider only, with no provider suffix. Type to search, move with Up/Down, Enter to pick. Some models ask a second question for thinking level.



```

qcode - /model picker

Models | Type to filter | Up/Down, Enter to select

> qwen2.5-coder:7b
  qwen3:8b
  gemma3
  gpt-oss:20b

List shows the current provider's model names only.
Thinking: off / low / medium / high / max (model decides list)

```

Figure 4: `/model picker`. Model names only; the list comes from the current provider. An optional thinking level follows (off/low/medium/high/max). Choices depend on the model.

Providers you can use

- Ollama (local default): pull a model first, then `qcode --model qwen2.5-coder:7b`. Thinking uses the model's native think setting.
- OpenAI-compatible: `export OPENAI_API_KEY=...` then `qcode --provider openai --model gpt-5`. Thinking uses reasoning effort where the model offers it.

- OpenCode Go: ``export QCODE_API_KEY=...`` then ``qcode --provider opencode-go --model kimi-k3``. Options vary per model.
- Flags can also come from `QCODE_PROVIDER`, `QCODE_MODEL`, `QCODE_THINKING`, `QCODE_BASE_URL`, `QCODE_API_KEY`.

```
qcode --provider ollama --model qwen3:8b --thinking high "reply with OK"
qcode --provider opencode-go --model deepseek-v4-flash --thinking high "reply with OK"
```

Tip

On ``main``, `/model` and `/maxsteps` remember your choice. Other tabs, demo, and one-shot runs keep changes for that session only.

One-shot and demo

```
qcode "explain this repository"
qcode --json-events "run the tests" 2>events.jsonl
qcode --demo "show me how qcode works"
```

Pass a prompt to run once without the interactive screen. Assistant text goes to stdout and short progress events go to stderr. Demo mode needs no model and touches no files.

6. Working with multiple agents

Use extra agents to do independent jobs at the same time, for example one writing code while another reads tests. Each tab keeps its own history, draft, queue, model, and tool choices.

```
qcode - /agent list

Agents (up to 20) | Enter switches, Ctrl+C leaves

> main      qwen2.5-coder:7b  idle
agent-1     gpt-5            working
agent-2     kimi-k3          1 queued

/agent new | /agent switch <id> | /agent rename | /agent close
Each tab keeps its own history, draft, queue, and model.
```

Figure 5: `/agent list`. Enter switches to the highlighted agent.

Command	What it does for you
<code>/agent</code>	Create an agent.
<code>/agent list</code>	Pick and switch with Up/Down + Enter.
<code>/agent switch <id></code>	Jump directly to one agent.
<code>/agent rename</code>	Give the current tab a clearer name.
<code>/agent cancel</code>	Stop that agent's running work.
<code>/agent close</code>	Remove that agent tab.

- Switch with `Ctrl+PageUp/PageDown` (`Alt+,` and `Alt+.` also work).

- Ask the main agent to split work and it can coordinate helpers; you can also create tabs yourself with `/agent`.
- Queues are per agent (up to 16 waiting each). A slow helper never blocks ``main``.

7. Planning before coding

Plan mode investigates without changing files. Enter it while idle, let qcode study the code, answer any design questions, review the submitted plan, then build it.

```

qcode - Plan mode

(Plan)> investigate retry for the login flow
PLAN in status bar: read-only investigation, no file changes

Plan submitted: summary + steps + checks
  1. Add retry helper with backoff
  2. Cover timeout + invalid token cases
  3. Validate: run focused tests

/plan show review | /plan act build it | /plan off leave

```

Figure 6: Plan mode. The prompt shows (Plan) and the status bar shows PLAN until you act or leave.

1. Type ``/plan`` to enter. The status bar shows PLAN.
2. Describe the task. Answer focused questions if qcode asks (you can type a custom answer).
3. Type ``/plan show`` to re-read the latest plan any time.
4. Type ``/plan act`` to build the latest plan, or ``/plan off`` to leave without building.

Tip

``/new`` clears the saved plan. Plan commands need an idle agent with no running or queued work.

8. Skills and SkillPlan

Using skills

Skills are reusable instruction bundles, for example a team review checklist. Turn them on per tab with ``/skill``. Only selected skills are shared with the assistant.

- Type ``/skill`` to see names plus one-line summaries. Space selects, Enter applies.
- Your account, workspace, and configured folders can each provide skills.
- New files appear in ``/skill`` without restarting.
- A ready-made collection ships with qcode under docs/skills. Enable only what helps.

Creating a skill safely

Use ``/skillplan`` to turn a rough idea into a reviewed skill. It asks about purpose, workflow, inputs, outputs, and limits, then saves a draft for approval.

```
/skillplan review database migrations before deployment
```

- ``/skillplan show`` reviews the draft. ``/skillplan create`` writes it. ``/skillplan off`` leaves without writing.
- After creation, run ``/skill`` to enable the new skill.
- SkillPlan can read context but cannot run shell commands or change files until you approve creation.

9. Remembering preferences with `/learn`

Use ``/learn`` after a good answer to save a durable preference or reusable procedure. qcode shows a preview and asks for approval. Nothing is saved until you answer ``y`` or ``yes``.

Command	What it does for you
<code>/learn</code>	Propose memory from recent conversation.
<code>/learn list</code>	Show saved items and IDs.
<code>/learn forget <id></code>	Review then delete one item.
<code>/learn compact</code>	Merge duplicates (asks first, backs up).

- Memory is per user on this device and shared across folders.
- Keep entries reusable: state the language or condition when they apply.
- Review proposals: secrets are rejected, but you decide what is worth keeping.

10. Web fetch and search

Web tools read public pages and search results as text. They start OFF. Turn them on for the current session with ``/tool``.

- Type ``/tool`` and enable ``web_fetch`` and/or ``web_search`` independently.
- ``/new`` or restarting turns them off again. One-shot runs keep them off.
- Use them for docs and public references. They do not run page scripts and do not open a browser.

Tip

If a page needs login or runs an app, fetch will not see it. Paste the relevant text into qcode instead.

11. Sessions: autosave, resume, and export

- Sessions autosave while you work and on clean exit. Each launch starts its own session.
- Type ``/resume`` to reopen a session for this folder. Entries show a preview and age, newest first. Finish or cancel running work before switching.
- Empty and demo/one-shot runs are not saved. ``/new`` resets only the current tab inside its session.
- After a crash, qcode reopens the last good checkpoint and marks unfinished work interrupted. It never reruns tools by itself.
- Sessions live outside your project (for example `~/local/state/qcode/sessions` on Linux). They keep private file permissions.

12. Controlling qcode from a browser

Type `/remote` to drive the same session from a phone or another browser. Pick Pure Web for a trusted local network or Tailscale for your tailnet. Avoid the `No auth` option unless you truly want an open short-lived demo.

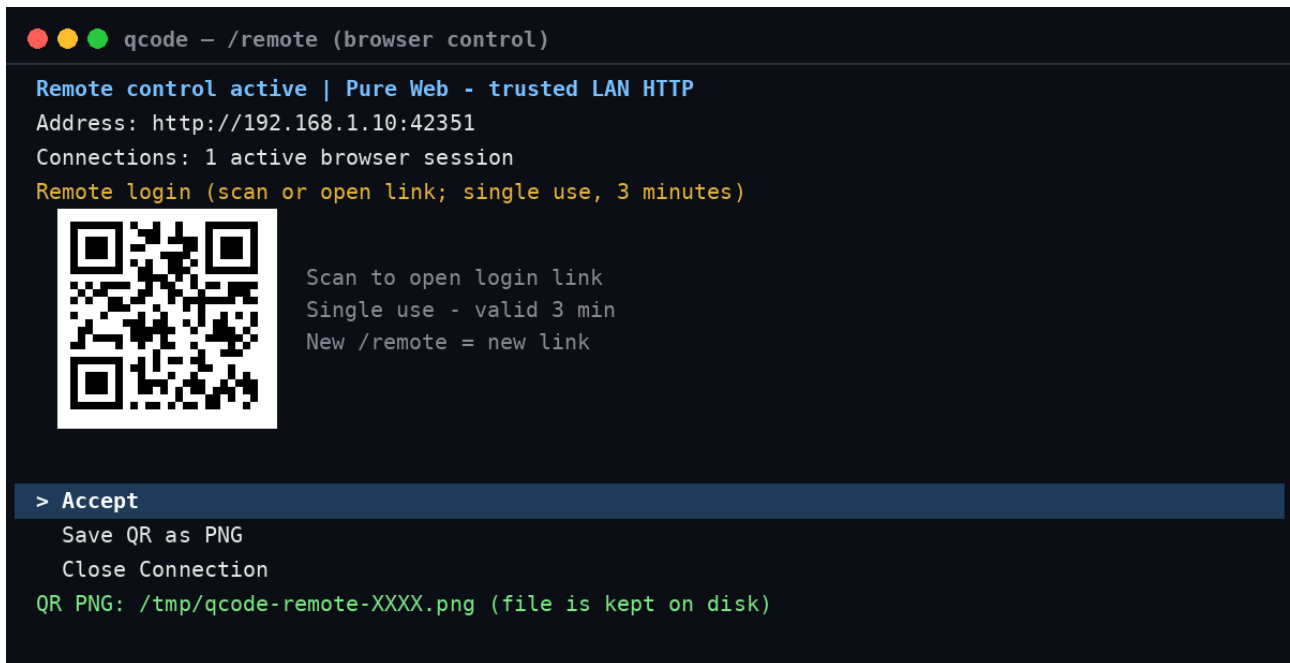


Figure 7: `/remote` screen. Scan the code or open the link, then Accept. Save QR as PNG if the code is too tall for your window.

1. Type `/remote` and choose how to connect. For multiple networks, pick the interface to share.
2. Scan the QR or open the shown link on the other device. Each link is single-use and lasts 3 minutes.
3. If the code does not fit, choose `Save QR as PNG` and open the shown file path. The file is kept on disk.
4. Choose Accept to keep the connection open. Run `/remote` again for one more device; existing browsers stay connected.
5. To stop all browsers, choose Close Connection and confirm, or quit qcode.
 - The browser shows the same tabs, queues, prompts, questions, and selectors as the terminal.
 - Reloading the same browser tab resumes. A fresh browser needs a fresh link.
 - In the browser, `/export` downloads the same HTML you get in the terminal.
 - Use only on networks you trust. Pure Web is plain HTTP on your LAN.

13. Settings you actually change

Most daily choices live in the terminal (`/model`, `/maxsteps`, `/statusline`, `/tool`). Use a config file only for defaults you always want.

```
provider = "ollama"
```

```
model = "qwen2.5-coder:7b"
max_steps = 32
sandbox = true
```

- Copy ``config.toml.example`` to your user config location. Missing files are ignored.
- Command flags beat environment variables. Both beat config files. Config files beat built-ins.
- Prefer environment variables for keys: `QCODE_API_KEY` or `OPENAI_API_KEY`. Keep key files private (mode 0600).
- Common knobs: thinking level, `agent_timeout` like "5m", `interactive` true/false, `statusline_hidden`, `web_search` backend, learning budget, skills paths.
- Restart after changing the web backend. ``/model`` and ``/maxsteps`` on ``main`` save automatically.

14. Troubleshooting and tips

Command	What it does for you
No colors?	Run with <code>NO_COLOR=1</code> . For ASCII borders use <code>QCODE_ASCII=1</code> .
Narrow window?	Status bar drops low-priority parts first, then wraps. Hide more with <code>/statusline</code> .
Slow session?	Try <code>/compact</code> , then <code>/new</code> for a fresh tab context.
Need detail?	Try <code>/verbose</code> . For step limits use <code>/maxsteps N</code> .
Lost output?	Scroll with PageUp/PageDown. <code>/history</code> finds old answers. <code>/export</code> saves HTML.
Remote blocked?	On Linux run once: <code>sudo tailscale set --operator=\$USER</code> . Then retry <code>/remote</code> .

Tip

Good defaults for most users: local model for drafts, stronger hosted model for final edits, Plan mode for risky changes, one agent per independent job.

15. Cheat sheet (copy this page)

Command	What it does for you
<code>/help</code>	Help for all or one command.
<code>/model</code>	Switch model by name only (+ thinking).
<code>/agent ...</code>	Create, list, switch, close tabs.
<code>/plan ...</code>	Plan, show, act, off.
<code>/skill</code>	Enable instruction bundles.
<code>/skillplan ...</code>	Draft and create a skill.
<code>/interactive</code>	Let normal work ask up to 3 questions per prompt.
<code>/learn ...</code>	Save/list/forget preferences.
<code>/tool</code>	Toggle web fetch/search.
<code>/resume, /new</code>	Reopen session / reset tab.
<code>/history</code>	Find and re-read answers.

/diff	Expand file-change preview.
/export	Save pretty or raw HTML.
/remote	Browser control via QR link.
/statusline	Choose status bar parts.
/maxsteps	Show/set step limit.
/clear, /quit	Redraw banner / leave.

One-shot recipes:

```
qcode --model qwen2.5-coder:7b "explain this repository"
qcode --provider openai --model gpt-5 "summarize the diff"
qcode --demo "show me how qcode works"
```

About this manual

This manual teaches use, not construction. It omits source layout, network internals, and developer extension points on purpose. For command details, type `/help`` inside qcode. Generated September 27, 2026 as version 1.0. Screenshots are illustrative examples rendered for print clarity.

Chapters in this manual

- How to use this manual
- 1. Getting started
- 2. Touring the terminal
- 3. Asking, queueing, and history
- 4. Slash commands at a glance
- 5. Choosing models and providers
- 6. Working with multiple agents
- 7. Planning before coding
- 8. Skills and SkillPlan
- 9. Remembering preferences with `/learn`
- 10. Web fetch and search
- 11. Sessions: autosave, resume, and export
- 12. Controlling qcode from a browser
- 13. Settings you actually change
- 14. Troubleshooting and tips
- 15. Cheat sheet